CS 251 Integrative Project 1 (3 cr) · 2026-2027 · Class 3 · Friday, October 9, 2026

Microeconomics Refresher: What Is an Hour Worth?

Principles of Microeconomics applied to the studio's time: opportunity cost, marginal analysis, the cost categories, break-even and payback. Phase 1 · Analysis · the hour before Brick 3.

Asen Grozdanov (Cyber), Cyber.SoHo Educational Hub

Lecture of one hour in four parts, an activity after each part, then the Brick 3 workshop

Educational material — Cyber.SoHo. Governed by the Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy, v2.0

The owner's question, and where we are

- “Which routine should we automate first, how much time would it really save, and is a free assistant good enough, or should we pay for one?” — the owner, Class 1
- Class 2 produced the problem statement, three research questions, user stories, a data dictionary (Brick 2, due last night)
- Today: the microeconomics of the studio's time; Class 4: the statistics of the time log; Brick 4 closes Phase 1
- No new theory: Principles of Microeconomics pointed at one file (boreal_routines_2026.csv) and one owner
- Today's four lessons are the method of research question 2; Brick 3 this afternoon is its first answer

The three given numbers

28 $ per hour — loaded hourly value of staff time

20 $ per user per month — the paid assistant tier

the free API quota — posted on the LMS with the date it was read

Everything else a formula needs is a stated assumption, written at the top of the script

What you will be able to do at the end of the hour

1. Compute the monthly cost of a routine from its minutes, the hourly value and its runs per month, and state the opportunity cost of doing it by hand.

2. Compute the marginal benefit and the marginal cost of automating one more routine, apply the rule MB ≥ MC, rank the candidates by net benefit and recognize diminishing returns.

3. Classify a cost as fixed, variable or sunk with respect to a stated quantity, and compute total, average and marginal cost, including the kink of a free quota.

4. Compute the break-even hours of a paid tier, the payback period of a set-up and a make-or-buy comparison over a horizon, and name a decision candidate backed by two numbers.

5. Map every formula of the class to a column of the data file, a given number or a stated assumption, and to the EconEngine method that will implement it.



Plan of the hour: four parts, four activities, then Brick 3

Minutes	Part	What you will do
0:00–0:02	Opening	Parking Lot answers from Class 2; where we are; the question of the hour
0:02–0:11	Lesson 3.1	Scarcity, choice and opportunity cost; the cost of a routine; the loaded hourly value and the next-best use
0:11–0:15	Activity 1	Visualization: the monthly bill of e-mail triage on the board (≈ 1 633 $); Code 3.1.A confirms it on the file
0:15–0:26	Lesson 3.2	Marginal benefit and marginal cost of automating one more routine; the MB ≥ MC rule; ranking; diminishing returns
0:26–0:30	Activity 2	Short practical: rank three routines by net benefit; compare with a neighbour; Code 3.2.A
0:30–0:41	Lesson 3.3	Fixed, variable, total, average and marginal cost; the free quota; the subscription; sunk cost
0:41–0:45	Activity 3	Fun quiz (4 questions) then a 2-minute zen break
0:45–0:56	Lesson 3.4	Break-even hours, payback months, make-or-buy; every formula mapped to a column and to an EconEngine method
0:56–1:00	Activity 4	Flash cards (8): opportunity cost, MB, MC, FC, VC, AC, break-even hours, payback
1:00–1:10	Workshop brief	Brick 3, Economic Analysis Script: what to build in the two hours, the checkpoint at 2:40

The running example: Boréal Studio and its file

- Fictional five-person consultancy of Montréal; six routines: e-mail triage, stand-up notes, scheduling, invoice checks, social posts, daily report
- boreal_routines_2026.csv: 14 working days (Sep 14 – Oct 1, 2026), 420 runs, one row per run; nine columns
- Two deliberate events: the daily-report template from day 8; assistant-triaged e-mail for S1–S3 from day 10 (tool = jarvis)
- Columns used today: minutes, tool, routine, staff_id, date; the given numbers are not columns
- Teaching copy of the file in the package; the LMS copy is the one your Brick uses

Stated assumptions (Table 3.1.A)

Billable rate (next-best use): **65 $/h**

Price per request beyond the quota: **0.02 $**

Set-up of the e-mail tool: **40 h**; maintenance **1 h/month**

Review of the assistant's work: **2 min per run**

Planning horizon: **12 months**

Free quota in the examples: **3 000 requests/month**

Scarcity forces a choice; a choice has two sides

- **Scarcity**: less of a resource than people would like to use; for the studio, the scarce resource is **staff time**
- Five people, about eight hours a day: an hour on e-mail is an hour not on a client
- **Microeconomics** studies these choices one decision-maker at a time: a household, a firm, an owner
- The owner's words are microeconomic words: “first” = a ranking; “how much” = a measurement; “good enough” = a comparison
- Research question 2 of Brick 2 is today's method; Brick 3 is its first answer; EconEngine (Phase 2) is its code

The cost of a routine: minutes become dollars

cost per month = minutes per run ÷ 60 × hourly value × runs per month

- E-mail triage: about 35 minutes a run by hand; 5 staff × 20 working days = 100 runs a month
- 35 ÷ 60 × 28 × 100 = **1 633.33 $ per month** (Activity 1 computes it on the board; Code 3.1.A finds 1 640 $ on the file)
- The six routines together: the **routine bill**, about 5 800 $ and 208 staff hours a month; e-mail triage is 28 % of it
- Write the hourly value once, as a named constant at the top of the script; never repeat the number inside formulas

Where the hourly value comes from: the loaded cost

- **Loaded hourly cost** = the salary for the hour **plus the employer's overhead** for it: contributions, vacation pay, equipment, the office
- It is what the hour costs the studio, not what the employee sees on a pay slip
- A real figure starts from published wage data (for example Statistics Canada's earnings tables) and adds the overhead of the business
- For the project: **28 $ per hour**, a given number; use it everywhere; it is an input, not a fact of nature
- The same minutes give a different bill at 28 $ and at 65 $: your program takes the rate as a parameter and your notes say which one you used

Opportunity cost: the value of the next-best use

- The 1 633 $ is an **explicit cost**: money the studio pays for the hours spent on e-mail
- The **opportunity cost** of an hour is the value of its **next-best use**, the best alternative given up
- If the hour could be billed to a client at 65 $, manual e-mail triage gives up 65 $ of revenue, whatever the salary was
- This is why “free” software still costs something: learning, setting up and checking it take hours with a next-best use
- Brick 3 values the team's set-up hours at the same 28 $/h for exactly this reason
- The studio's 208 routine hours: about 5 800 $ at the loaded cost, about 13 500 $ of client work at the billable rate — two correct answers to two different questions

Activity 1: the monthly bill on the board

- On the board: e-mail triage, 35 minutes a day, 5 staff members, 20 working days, 28 $ per hour
- Write the formula first, then the numbers, then the result
- Then: what changes if the hourly value is 65 $? If the routine takes 25 minutes?
- Run Code 3.1.A on the file to confirm the number (the file gives 35.15 minutes and 1 640 $)

Expected

35 ÷ 60 × 28 × 100 = **1 633.33 $**

At 65 $/h: 3 791.67 $

At 25 minutes: 1 166.67 $

The file: 1 640.12 $ (35.15 min, not 35)

Watch at home: opportunity cost (under 15 minutes each)

5:48 length · Khan Academy

Opportunity Cost

Why watch it: Defines opportunity cost as the value of the next-best thing given up and works one example with a production-possibilities curve. Watch it for Part 1 of this lesson: every choice about an hour has a cost even when no money is paid.

Watch on YouTube Creator page

12:00 length · CrashCourse (Jacob Clifford and Adriene Hill)

Intro to Economics: Crash Course Economics #1

Why watch it: An overview of scarcity, the choices it forces, opportunity cost and incentives, with everyday examples. Watch it for the framing of the whole lesson before the studio's numbers.

Watch on YouTube Creator page

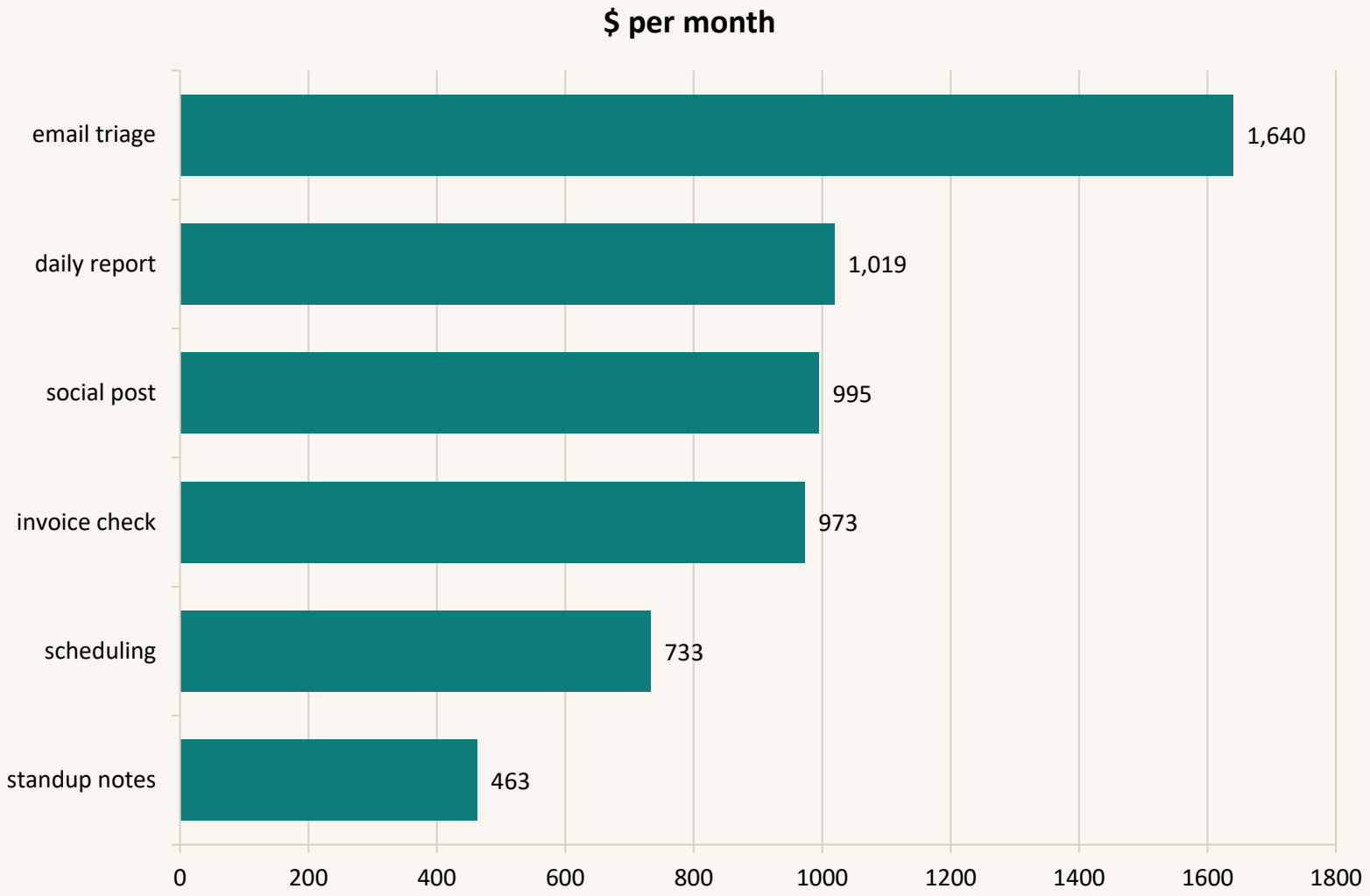
Each link was confirmed online and public on October 8, 2026; lengths from the creators' pages or chapter markers, all under 15 minutes. External videos: no affiliation, their own terms apply (Legal Notice sections 5, 14 and 15). Check any address with VirusTotal before you open it.

Real-life example 3.1: the studio's monthly routine bill

- **Task:** what each routine costs per month as it is done today; the total in hours and dollars; the same hours at the next-best use
- Read the file with csv.DictReader; convert minutes with int(); skip the 15 pilot runs (they are an experiment, not the current practice)
- Group by routine: total minutes ÷ runs = average minutes per run; 100 runs a month per routine
- Result: e-mail triage 1 640 $, daily report 1 019 $, social posts 995 $, invoice checks 973 $, scheduling 733 $, stand-up notes 463 $
- Total 5 821 $ and 208 hours a month; at 65 $/h the same hours are 13 514 $ of client work not done
- **Brick 3:** the per-routine and per-staff table and the routine bill (analysis/econ_sheet.py, docs/econ_output.txt); routine_cost() docstring



The monthly routine bill, dollars per month (Code 3.1.A)



- Total: **5 821 $** and **208 h** a month
- E-mail triage: 1 640 $, 28 % of the bill, 35.1 min a run
- Every routine: 100 runs a month (5 staff × 20 days)
- At the billable rate (65 $/h) the same hours are **13 514 $** of client work
- The largest line is the first automation candidate; Lesson 3.2 ranks it against the others

CS 251 Integrative Project 1 · Class 3 · Cyber.SoHo Educational Hub · © 2026

Code 3.1.A: class03_routine_bill.py, step by step

1. Lines 1 to 17 are the header: the task, the link with the lecture and the data description. Every line that starts with # is a comment ignored by Python; each code line below is preceded by one.

2. Three imports: csv reads the file safely, Path finds the data file next to the script without any absolute path, and defaultdict creates a missing group automatically.

3. The settings: the hourly value (28 $/h, loaded), the billable rate of the next-best use (65 $/h, owner's estimate), the 20 working days of a month, the five staff members, the file name, and the VIEW dictionary that only the lecture page reads.

4. find_file() looks for the data file next to the script, in data/ and in ../data/, and stops with a clear message when it is absent. load_rows() reads the file with csv.DictReader and converts minutes and done to integers: a CSV file contains text only.

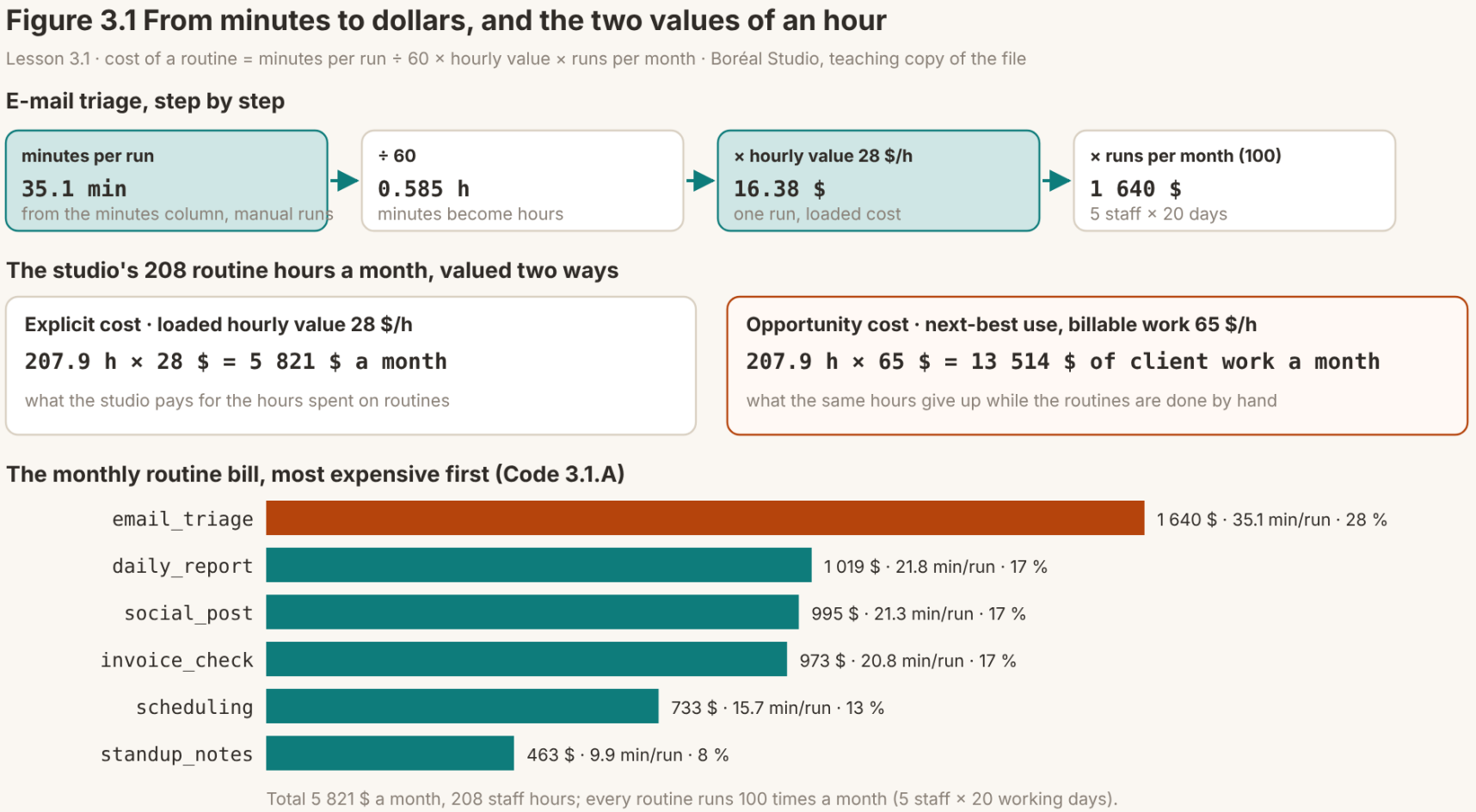
5. routine_cost() is the formula of the lesson in one line: minutes / 60 × hourly value × runs per month. opportunity_cost() is hours × the rate of the next-best use.

6. Step 1 reads the 420 rows and prints how many were read. Step 2 groups them by routine: each row adds one run and its minutes to its routine's group; the pilot rows of the assistant are skipped, because the bill describes the current practice.

7. Step 3 processes every routine: average minutes per run, runs per month (five people, once a day, 20 days = 100), the monthly cost with routine_cost(), the hours per month; the studio totals grow with each routine, and the visualization adds one bar.

8. Step 4 sorts the routines by cost and prints the bill, most expensive first, with the share of each routine; the STUDIO TOTAL line is the monthly routine bill: about 5 821 $ and 208 hours.

Figure 3.1: from minutes to dollars, and the two values of an hour



- The chain: minutes → hours → $ per run → $ per month
- Loaded cost 28 $/h: **5 821 $** a month
- Next-best use 65 $/h: **13 514 $** of client work
- Bars: the bill, most expensive first

Figure 3.1 The chain minutes per run → hours → dollars per run → dollars per month, the two valuations of an hour (loaded cost 28 $/h, next-best use 65 $/h) and the monthly routine bill of the six routines computed by Code example 3.1.A.

Practice 3.1: flash cards and self-check (formative)

Flash cards: can you answer without looking?

- 1. What does scarcity mean, and what is the scarce resource of Boréal Studio?
- 2. Write the formula for the monthly cost of a routine.
- 3. What is a loaded hourly cost?
- 4. Define opportunity cost.
- 5. Why does 'free' software still cost something?

Self-check on the page: 4 MCQ + 2 MSQ (formative, not graded)

- Q1 MCQ: The daily report takes 22 minutes a run, each of the five staff members writes o…
- Q2 MCQ: Which statement describes the loaded hourly cost of 28 $?
- Q3 MCQ: A consultant spends an hour sorting e-mail instead of billing a client 65 $ for…
- Q4 MCQ: Why does Code example 3.1.A leave the fifteen pilot runs of the assistant out of…
- Q5 MSQ (select 3): Which of the following are inputs of the cost-of-a-routine formula? (Select 3)
- Q6 MSQ (select 2): Which two statements about opportunity cost are correct? (Select 2)

Thinking at the margin: the marginal benefit (MB)

MB per month = minutes saved per run ÷ 60 × hourly value × runs per month

- A decision about **one more unit** (one more routine automated, one more level) is a decision **at the margin**
- **Marginal benefit (MB)**: the value of the time the automation saves per month
- Minutes saved = minutes of a manual run − minutes of an assisted run − minutes a person still spends reviewing the assistant's work
- E-mail pilot: 35.9 − 13.7 = 22.1 minutes; net of 2 review minutes: 20.1; 100 runs a month at 28 $/h: **MB = 938 $ a month**

The marginal cost (MC) of automating a routine

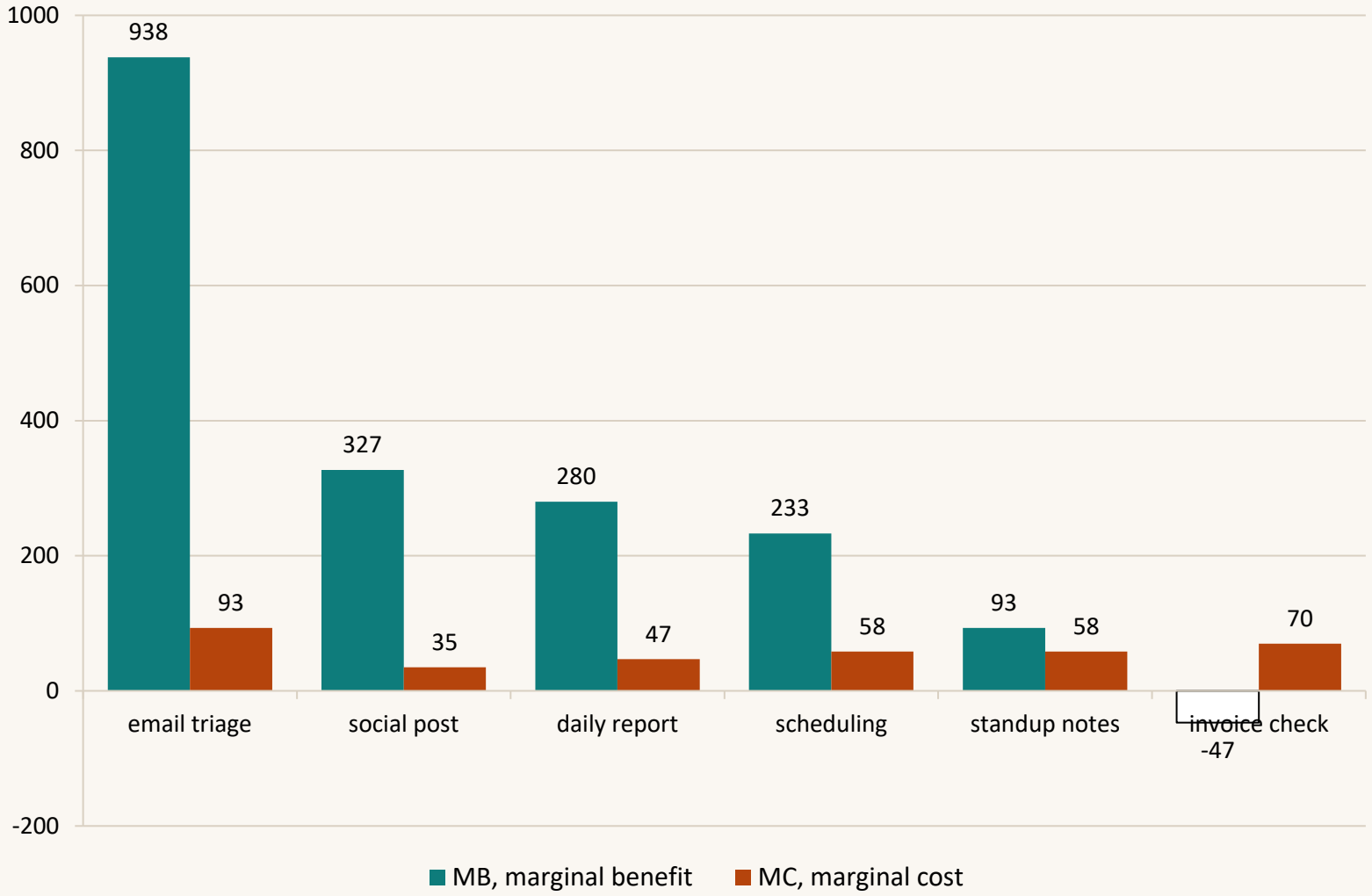
MC per month = paid requests × price per request + set-up cost ÷ horizon in months

- **Marginal cost (MC)**: what that one more automation costs per month; two parts
- Requests to the language-model **API** (Application Programming Interface): **zero inside the free quota**, the price per request beyond it
- A monthly share of the **set-up**: the team's hours × the hourly value, spread over the planning horizon (12 months)
- E-mail tool: 300 requests a month (inside a quota of 3 000) → 0 $; set-up 40 h × 28 $ = 1 120 $ → 93.33 $ a month; **MC = 93.33 $**
- Both numbers are in dollars per month, so they can be compared

The rule: automate while MB ≥ MC; rank by MB − MC

- **Automate while MB ≥ MC**: if the extra benefit covers the extra cost, the studio is better off; if not, it is not, whatever the routine's total cost was
- **Net benefit = MB − MC**; ranking the candidates by net benefit answers the owner's word “first”
- E-mail triage: 938 − 93.33 = **844.67 $ a month: automate**
- The invoice check fails: the assistant saves 5 minutes, a person then verifies each flagged invoice for 6 minutes → negative net minutes → **do not automate**, for now
- Ranking of the five that pass: e-mail triage 845 $, social posts 292 $, daily report 233 $, scheduling 175 $, stand-up notes 35 $

MB against MC for the six candidates, dollars per month (Code 3.2.A)



- Rule: automate while **MB ≥ MC**
- Five pass; the invoice check does not (MB = −47 $)
- Net benefit = MB − MC gives the order: e-mail, social, report, scheduling, stand-up
- All savings except e-mail triage are stated assumptions; change them at the top of the script and the ranking updates

Diminishing returns: each added unit brings less than the one before

- Across routines: the cumulative net benefit of the ranking flattens: +845, +292, +233, +175, +35 $ a month
- Three students, fourteen weeks: the fourth tool costs the same effort as the first and returns far less → “three tools that work beat ten that do not”
- Inside one routine: levels of e-mail triage: sort and label (15 min saved a run), draft the replies (5 more), send without review (2 more)
- Each level is judged on its **own** extra MB and extra MC: 700 vs 47 $ (add), 233 vs 58 $ (add), 93 vs 105 $ (**stop here**; plus a risk no spreadsheet shows)
- **Stop at the first level where the extra MB no longer covers the extra MC** — the same rule, applied one unit at a time



Activity 2: rank three routines by net benefit

- Three candidates, 100 runs a month each, 28 $/h, set-up spread over 12 months, requests inside the quota
- (A) daily report: 6 net minutes saved a run, 20 set-up hours
- (B) scheduling: 5 net minutes, 25 hours
- (C) social posts: 7 net minutes, 15 hours
- Compute MB, MC and the net benefit of each; rank them; which one would you build first? Compare with a neighbour

Expected

MB = min ÷ 60 × 28 × 100; MC = hours × 28 ÷ 12

(A) 280.00 − 46.67 = **233.33 $**

(B) 233.33 − 58.33 = **175.00 $**

(C) 326.67 − 35.00 = **291.67 $**

Order: C, A, B → build the social-post tool first

Watch at home: marginal analysis (under 15 minutes each)

5:08 length · Khan Academy

Optimal decision-making and opportunity costs (AP Microeconomics)

Why watch it: Shows a decision-maker comparing the marginal benefit of one more unit with its marginal cost, including the opportunity cost of time, and continuing only while the benefit covers the cost. It is the shortest statement of the MB >= MC rule of Part 2.

Watch on YouTube Creator page

11:00 length · CrashCourse

Marginal Analysis, Roller Coasters, Elasticity, and Van Gogh: Crash Course Economics #18

Why watch it: Explains thinking at the margin and diminishing marginal utility: each extra unit is worth a little less. Watch the first half for the diminishing-returns part of this lesson; the elasticity part is not needed here.

Watch on YouTube Creator page

Each link was confirmed online and public on October 8, 2026; lengths from the creators' pages or chapter markers, all under 15 minutes. External videos: no affiliation, their own terms apply (Legal Notice sections 5, 14 and 15). Check any address with VirusTotal before you open it.

Real-life example 3.2: which routine should JARVIS automate first?

- **Task:** measure the e-mail saving from the pilot rows, state the other savings, compute MB and MC for six candidates, apply the rule, rank, test three levels
- The pilot participants are compared with themselves: 35.9 min by hand, 13.7 with the assistant → 22.1 saved, 20.1 net of review
- Five other routines: **stated assumptions** in a table at the top of the program, where the owner can change them
- Five routines pass the rule; the invoice check does not (a 6-minute human check follows every flagged invoice)
- Levels of e-mail triage: sort and label (add), draft the replies (add), send without review (stop)
- **Brick 3:** the automation-economics block for ≥ 2 routines (minutes saved, MB, set-up cost, MC at the free tier and at the paid price, net, rank); marginal_benefit() docstring

Code 3.2.A: class03_marginal_ranking.py, step by step

1. Lines 1 to 22 state the task, the link with the lecture and the data: the e-mail saving is measured from the pilot rows; everything else is a stated assumption that the owner can change.

2. The settings: hourly value, 100 runs per month per routine, the free quota (3 000 requests a month) and the price per request beyond it (0.02 $), both teaching assumptions to replace with the number read on the LMS, the 12-month horizon, and the table of candidates: minutes saved, review minutes, requests per run, set-up hours.

3. measured_saving() reads the file, keeps the e-mail triage rows, groups the minutes by (staff member, tool), finds who took part in the pilot, and compares those people with themselves: manual mean minus assisted mean.

4. marginal_benefit() is the MB formula; marginal_cost() is the MC formula with its two parts: paid requests beyond the quota times the price, plus the set-up cost divided by the horizon.

5. Step 1 prints the measurement: 35.9 minutes by hand, 13.7 with the assistant, 22.1 saved; the measured value replaces the None of the candidate table.

6. Step 2 evaluates every candidate: net minutes (saved minus review), MB, requests per month, MC, net benefit and the verdict of the rule; the visualization adds one pair of bars per candidate. The invoice check fails: its net minutes are negative.

7. Step 3 keeps the routines that passed, sorts them by net benefit and prints the ranking with a cumulative column: each line adds less than the previous one, which is diminishing returns across routines.

8. Step 4 judges three levels of e-mail triage on their own increments: extra minutes, extra MB, extra set-up share; the first two levels pass, the third does not, and the program stops there, as the rule says.

Figure 3.2: MB against MC, the rule, and diminishing returns

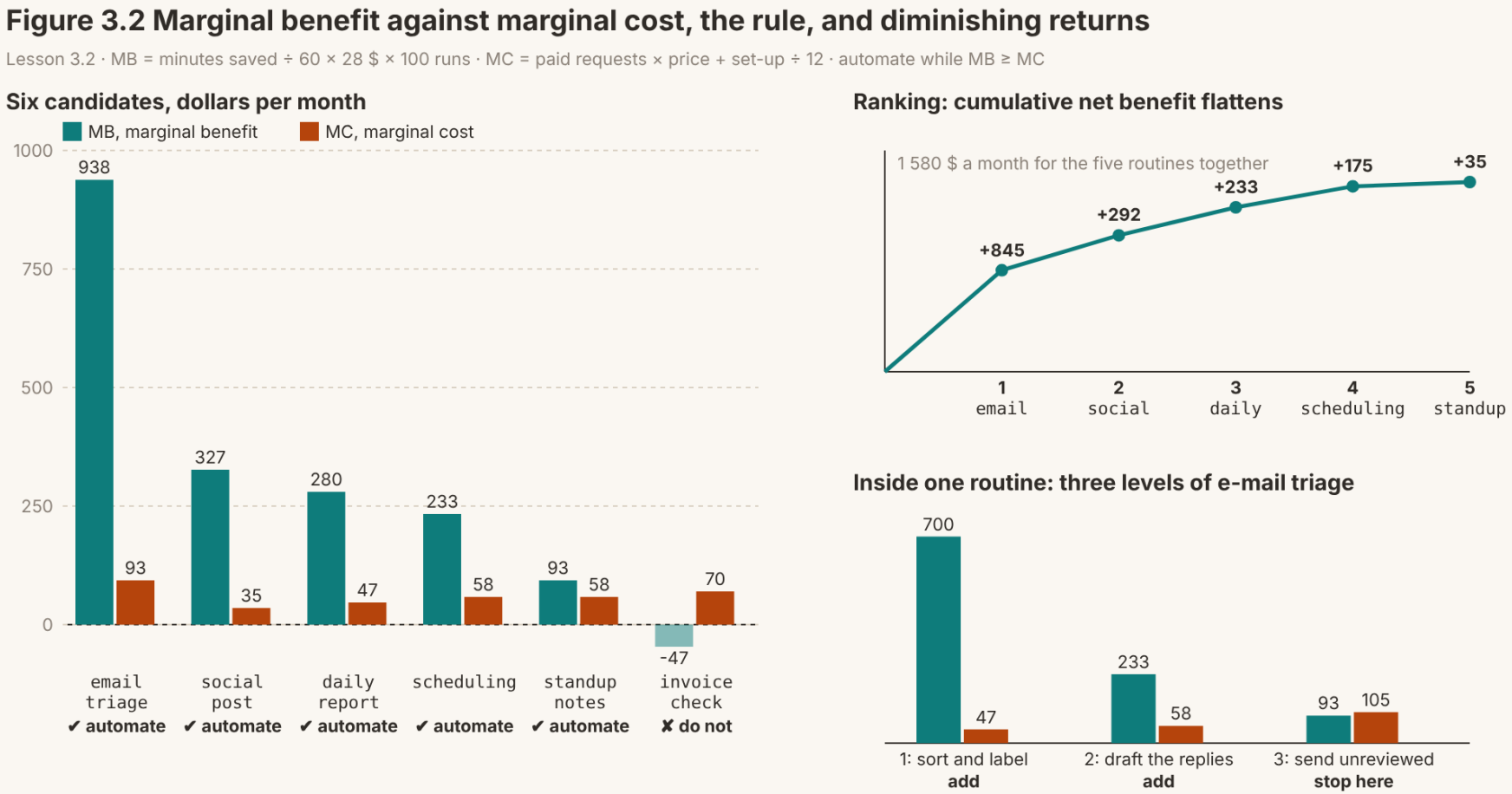


Figure 3.2 Left: MB and MC per month of the six candidate routines, with the rule MB ≥ MC applied (the invoice check fails). Right: the cumulative net benefit of automating the routines in rank order flattens, and the three levels of e-mail triage show where the extra MC overtakes the extra MB. Numbers from Code example 3.2.A.

- Left: MB and MC of the six candidates; the invoice check fails
- Top right: the cumulative net benefit flattens
- Bottom right: three levels of e-mail triage; level 3 stops

CS 251 Integrative Project 1 · Class 3 · Cyber.SoHo Educational Hub · © 2026

Practice 3.2: flash cards and self-check (formative)

Flash cards: can you answer without looking?

- 1. What does 'at the margin' mean?
- 2. Write the formula of the marginal benefit of automating a routine.
- 3. What are the two parts of the marginal cost of automating a routine?
- 4. State the decision rule of marginal analysis.
- 5. What are diminishing returns?

Self-check on the page: 4 MCQ + 2 MSQ (formative, not graded)

- Q1 MCQ: The assistant saves 9 minutes a run on social posts, a person spends 2 minutes c…
- Q2 MCQ: A routine's automation has MB = 233 $ a month and MC = 58 $ a month. Another has…
- Q3 MCQ: Why does the invoice check fail the rule in Code example 3.2.A although the assi…
- Q4 MCQ: In the cumulative column of the ranking, each added routine increases the total…
- Q5 MSQ (select 3): Which of these numbers belong to the marginal cost of automating a routine, as d…
- Q6 MSQ (select 2): Which two statements about the levels of e-mail triage automation are correct? (…

CS 251 Integrative Project 1 · Class 3 · Cyber.SoHo Educational Hub · © 2026

Fixed and variable costs: fixed with respect to what?

- **Fixed cost (FC)**: does not change with the quantity in the period: the paid tier, 20 $ per user per month = 100 $ for five seats, whatever the usage
- The team's set-up time is fixed too, but **paid once**: 40 h × 28 $ = 1 120 $, not a monthly cost
- **Variable cost (VC)**: changes with the quantity: requests beyond the free quota (price per request each); the staff minutes of every routine run
- Inside the free quota the variable cost of a request is zero, then it rises at the price per request: the cost curve has a **kink** at the quota
- **Total cost: TC = FC + VC**; compare two options over the same period and the same quantity
- Always say “fixed with respect to what”: a subscription is fixed per request but variable per user (one more user, 20 $ more)

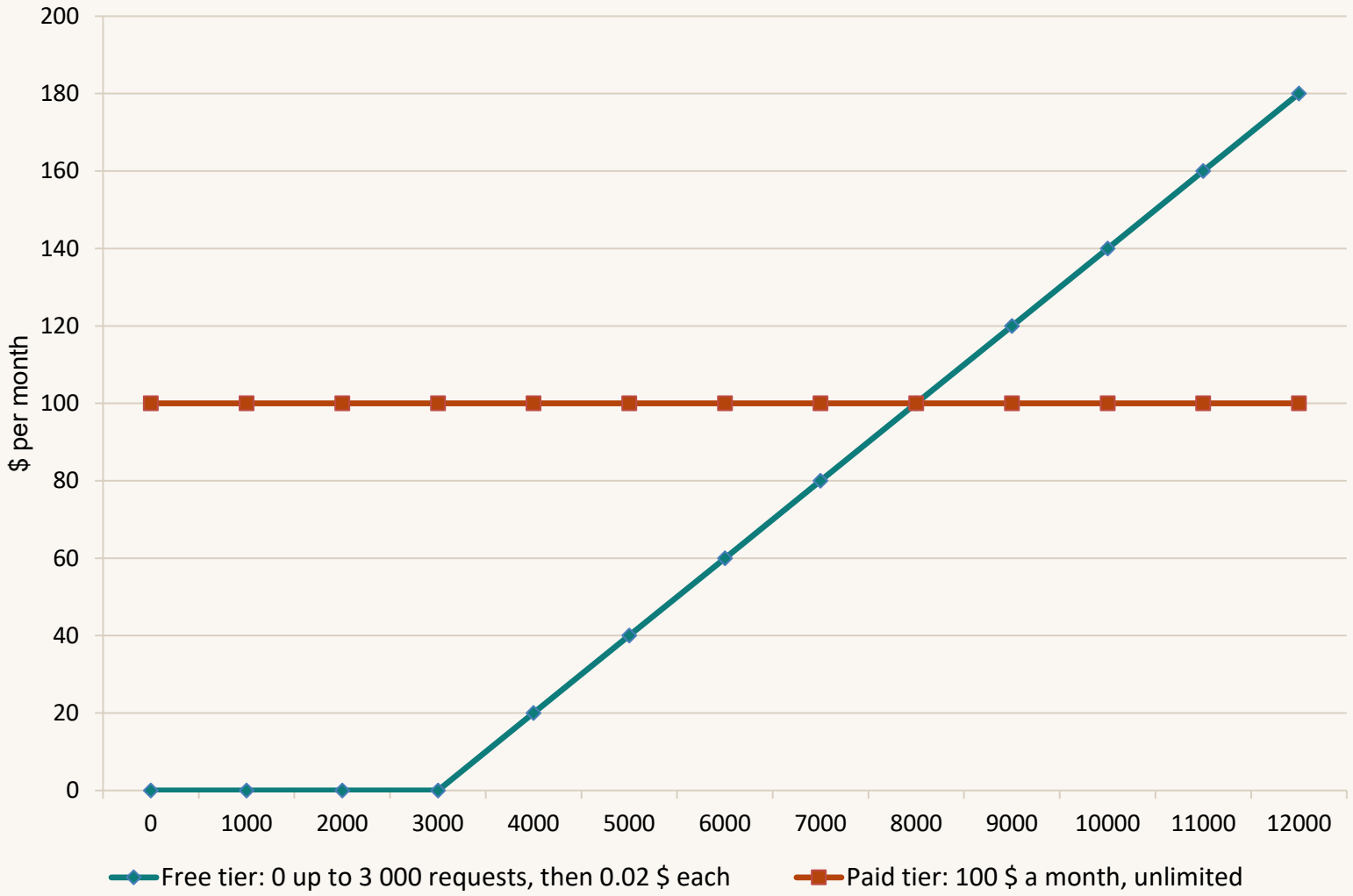


Average and marginal cost

AC = TC ÷ runs **MC = ΔTC ÷ Δruns (or per request)**

- **Average cost (AC)**: the cost per run; the paid tier's AC falls as the volume grows (100 $ spread over more runs); the free tier's is zero up to the quota, then rises slowly
- **Marginal cost (MC)**: the change in total cost for one more unit: **0 $ per request inside the quota**, 0.02 $ beyond (teaching assumption)
- The two total costs cross at 8 000 requests a month (quota + 100 $ ÷ 0.02 $); the studio needs 300 → the free tier wins by a wide margin, as long as the quota holds
- Units: AC per run, MC per additional run or request, FC per period or once; write the unit next to every number

Total cost per month of the two options as the volume grows (Code 3.3.A)



- Horizontal axis: requests per month
- Kink at the quota (3 000); crossing at **8 000**
- The studio: 300 requests → inside the quota, **MC = 0 $**
- Beyond the crossing the subscription costs less
- The quota is a number that can change: record it with its date (Brick 3)

CS 251 Integrative Project 1 · Class 3 · Cyber.SoHo Educational Hub · © 2026

The sunk cost: spent under every alternative, so it changes nothing

- **Sunk cost**: a cost already incurred that no present or future decision can recover: the 40 set-up hours
- The rule: **a sunk cost must not influence a decision**; a decision compares what *changes* from here on
- The trap: month one, benefit 938 $, running cost 28 $; someone subtracts the 1 120 $ of set-up: “−210 $, the pilot loses money, stop it”
- Stopping does not return the 1 120 $; it only forfeits 910 $ every month: continue (+910) against stop (0), the set-up in neither column
- Where the set-up belongs: before building, as a monthly share in MC (Lesson 3.2); after building, as the amount to recover in the payback (Lesson 3.4)
- EconEngine.payback_months() takes the set-up cost as a parameter; no method ever deducts it from a future saving

Activity 3: fun quiz (hands up), then two minutes of zen

- 1. Set-up time: fixed or variable cost?
- 2. A per-request price beyond the quota: fixed or variable?
- 3. “Break-even at 0.7 hours a month” means what, in plain words?
- 4. The set-up hours are already spent: do they change today's decision about continuing the pilot?
- Then: stand up, roll the shoulders, breathe in for four counts, hold four, out four, three times

Answers

1. **Fixed**, and sunk once spent
2. **Variable**: it grows with the requests
3. The paid tier pays for itself if it saves more than **42 minutes a month** per user
4. **No**: a sunk cost is the same under every alternative

Watch at home: cost categories and the sunk cost (under 15 minutes each)

5:17 length · Jacob Clifford (ACDC Econ)

Short-Run Costs (Part 1), Micro Topic 3.2

Why watch it: Builds a cost table step by step: fixed cost, variable cost, total cost, then marginal and average cost, and shows why the marginal cost rises with diminishing returns. It covers every cost term of this lesson in five minutes.

Watch on YouTube Creator page

about 6:00 length · Marginal Revolution University (Alex Tabarrok)

What Is Thinking on the Margin, and How Does It Solve the Sunk Cost Fallacy?

Why watch it: Explains why only the next unit's marginal benefit and marginal cost matter, then shows the sunk-cost error with everyday examples. It is the clearest beginner explanation of why the set-up hours already spent must not enter the decision of Part 3.

Watch on YouTube Creator page

Each link was confirmed online and public on October 8, 2026; lengths from the creators' pages or chapter markers, all under 15 minutes. External videos: no affiliation, their own terms apply (Legal Notice sections 5, 14 and 15). Check any address with VirusTotal before you open it.

Real-life example 3.3: what it costs to run the e-mail tool as the volume grows

- **Task:** the monthly cost table of the tool for 0 to 12 000 requests under the free tier and the paid tier; FC, VC, TC, AC, MC at every volume; the crossing; then continue or stop with the set-up treated as sunk
- Free tier: no fixed cost; VC = 0 up to the quota, 0.02 $ per request beyond; paid tier: 100 $ flat, no variable cost
- The lines cross at 8 000 requests; the studio needs 300: inside today's quota the free tier is costless per request
- Continue or stop? Forward-looking: 938 − 28 = +910 $ a month; the trap (−210 $) subtracts a sunk cost and reaches a false conclusion
- The sentence for the report and the Mid-Term: “the set-up cost is sunk; the decision to continue compares future benefits with future costs”
- **Brick 3:** the MC-per-run lines at the free tier and at the paid price; total_cost() and average_cost() docstrings with the zero-runs guard

Code 3.3.A: class03_cost_curves.py, step by step

1. Lines 1 to 21 state the task and the link with the lecture; no data file is needed: the volumes are generated by the program.

2. The settings: hourly value, 20 $ per user for five users, the free quota and the price per request (teaching assumptions), three requests per run, 100 runs a month, the 40 set-up hours already spent, the 20.1 net minutes saved, one maintenance hour a month, and the volumes 0, 1 000, ... 12 000.

3. Four small functions, one per definition: variable_cost() (paid requests beyond the quota × price), total_cost() (FC + VC), average_cost() (TC ÷ runs, with a guard that returns None when there is no run), marginal_cost() (ΔTC ÷ Δrequests).

4. Step 1 prints the fixed costs: 0 $ for the free tier, 100 $ a month for the paid tier, and the one-time set-up of 1 120 $, printed separately because it is not a monthly cost.

5. Step 2 is the table: for every volume, the runs it represents, VC, TC, AC and MC of the free tier, TC and AC of the paid tier; a dash marks an undefined average at zero runs; the visualization adds one point per line.

6. Step 3 computes the crossing volume (quota + 100 ÷ 0.02 = 8 000 requests) and compares it with the studio's 300; the sentence to remember is printed: inside the quota the marginal cost of a request is zero.

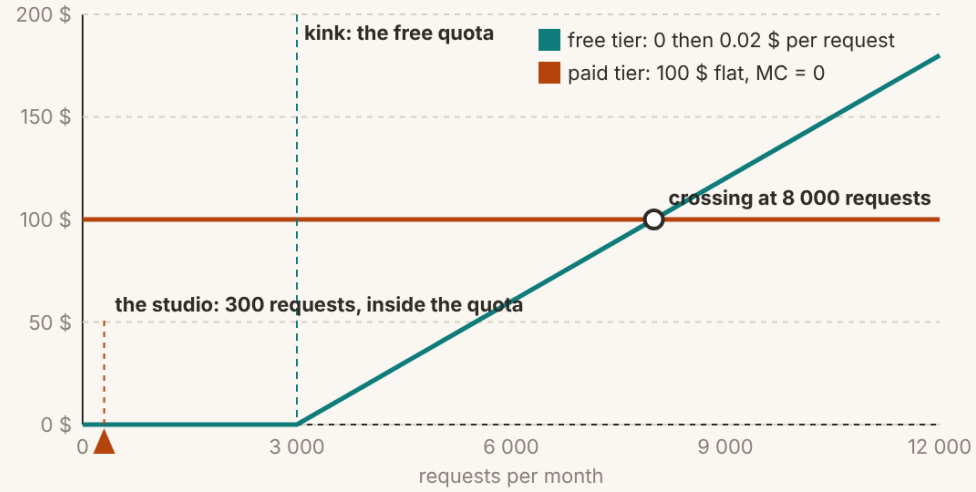
7. Step 4 is the decision: forward-looking benefit (938 $) against forward-looking cost (28 $), +910 $ a month: continue. Then the trap: subtracting the sunk 1 120 $ gives −210 $ and a false conclusion; the last line states the rule.

Figure 3.3: the cost curves of the two options, and the sunk-cost rule

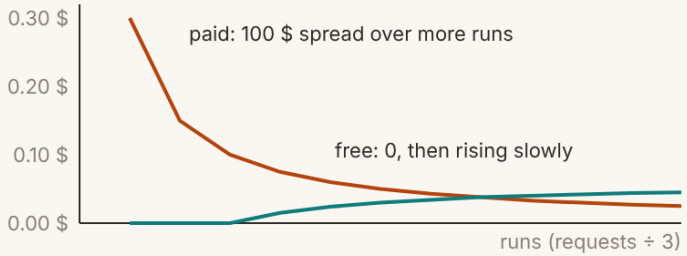
Figure 3.3 The cost curves of the two options, and the sunk-cost rule

Lesson 3.3 · TC = FC + VC · AC = TC ÷ runs · MC = ΔTC ÷ Δrequests · quota 3 000 requests, 0.02 $ beyond, paid tier 5 × 20 $ (teaching assumptions)

Total cost per month as the volume grows



Average cost per run



Continue or stop the pilot? Compare what changes

Continue	Stop
+ 938 $ saved	+ 0 $ saved
− 28 $ maintenance	− 0 $
= + 910 $ a month	**= 0 $ a month**

Set-up already spent: 1120 $ (40 h × 28 $)

Sunk: the same under both columns, so it enters neither.

The trap: 938 − 28 − 1 120 = −210 $ and the false conclusion 'the pilot loses money'. Stopping does not return the 1 120 $; it only forfeits 910 $ every month.

From the table of Code 3.3.A

requests	runs	VC free	TC free	AC free	MC free	TC paid	AC paid
0	0	0.00	0.00	-	-	100.00	-
3 000	1000	0.00	0.00	0.000	0.00	100.00	0.100
4 000	1333	20.00	20.00	0.015	0.02	100.00	0.075
8 000	2667	100.00	100.00	0.038	0.02	100.00	0.038
12 000	4000	180.00	180.00	0.045	0.02	100.00	0.025

Units: VC, TC in $ per month;
AC in $ per run; MC in $ per request.

Inside the quota MC = 0 $;
beyond it MC = 0.02 $.

Figure 3.3 Total cost per month of the free tier (zero up to the quota of 3 000 requests, then 0.02 $ per request) and of the paid tier (100 $ flat) as the volume grows, with the crossing at 8 000 requests and the studio's 300; the average cost per run of each option; and the two columns of the continue-or-stop decision, with the sunk set-up cost outside both. Numbers from Code example 3.3.A.

- Left: TC of the free tier (kink at 3 000) and of the paid tier (flat 100 $); crossing at 8 000; the studio at 300
- Top right: AC per run of both
- Right: continue vs stop, the set-up outside both columns

Practice 3.3: flash cards and self-check (formative)

Flash cards: can you answer without looking?

- 1. Define a fixed cost and give the two fixed costs of the project.
- 2. Define a variable cost and give two examples from the project.
- 3. Write the formulas of total, average and marginal cost.
- 4. What is a sunk cost, and what is the rule about it?
- 5. Where does the set-up cost legitimately enter the analysis?

Self-check on the page: 4 MCQ + 2 MSQ (formative, not graded)

- Q1 MCQ: The studio pays 20 $ per user per month for an assistant, whatever the number of…
- Q2 MCQ: Under the free tier, the quota is 3 000 requests a month and each request beyond…
- Q3 MCQ: At 2 000 runs a month, the paid tier costs 100 $ in total. What is its average c…
- Q4 MCQ: The team spent 40 hours (1 120 $) building the e-mail tool. This month the tool…
- Q5 MSQ (select 3): Which three statements about cost categories are correct? (Select 3)
- Q6 MSQ (select 2): Where does the set-up cost of a tool legitimately enter the analysis? (Select 2)

Break-even hours: when a paid seat pays for itself

break-even hours = monthly cost of the paid option ÷ hourly value

- The **break-even point** of an option: the quantity at which its benefit exactly covers its cost
- 20 $ ÷ 28 $/h = **0.714 h, about 43 minutes a month per user**; five seats: 100 ÷ 28 = 3.57 h a month
- The pilot saves 20.1 net minutes a run, 20 runs a month: **6.7 h per user**, nine times the break-even
- Answer to the owner: any assistant that does what the pilot did pays for its seat; choose on fit and risk, not on the seat price
- The formula contains no set-up hours (they belong to the payback); the guard: an hourly value ≤ 0 raises ValueError

Payback: how long the savings take to recover the set-up

payback months = set-up cost ÷ monthly net saving

- Set-up 1 120 $; monthly net saving 939 − 28 = 911 $ (the benefit of 20.1 net minutes over 100 runs, minus one maintenance hour) → **1.23 months**
- Special case, documented: a monthly net saving ≤ 0 → math.inf, “never”, not a crash
- Payback is simple and widely used; it ignores what happens after the recovery and does not discount the future → present it next to the monthly net saving
- Cumulative position: −1 120 $ at month 0, then +911 $ a month; the line crosses zero at 1.23 months and reaches +9 810 $ after a year

Make or buy: total cost over 12 and 24 months (Code 3.4.A)



- build = set-up + maintenance × months
- buy = subscription × months
- 12 months: buy (1 200 $) < build (1 456 $)
- 24 months: build (1 792 $) < buy (2 400 $)
- **The horizon decides**; so do fit (does the paid assistant do the task?) and the quota risk

CS 251 Integrative Project 1 · Class 3 · Cyber.SoHo Educational Hub · © 2026

The formula card: formula, columns, EconEngine method (Table 3.4.B)

Concept	Formula	Columns · givens · assumptions	EconEngine method (signature due with Brick 3)
Cost of a routine	min ÷ 60 × rate × runs/month	minutes, routine, staff_id, date; 28 $/h	routine_cost()
Opportunity cost	hours × next-best rate	hours; billable rate (stated)	opportunity_cost()
Marginal benefit	min saved ÷ 60 × rate × runs	minutes, tool (manual vs jarvis); review min	marginal_benefit()
Total / average cost	FC + VC; TC ÷ runs	quota, price; runs from date	total_cost(), average_cost()
Break-even hours	monthly price ÷ rate	20 $/user/month; 28 $/h	break_even_hours() (ValueError if rate ≤ 0)
Payback months	set-up ÷ monthly net saving	set-up hours logged in Phase 2; MB − running cost	payback_months() (math.inf if saving ≤ 0)

Tonight: the seven signatures with a docstring that states each formula, no body yet. Class 6: the bodies. Brick 9: the tests against Brick 3's numbers.

What these numbers can and cannot say

- They rest on 14 days, 5 people and 15 pilot runs, and on assumptions written at the top of a script
- Strong enough to rank the routines and to answer the owner's three questions with an order of magnitude
- Not strong enough to promise the second decimal
- Class 4 adds the tools to say how much the minutes vary and what 14 days can prove
- The recommendation of January carries both the numbers and their limits

Activity 4: flash cards, eight terms in pairs

- One asks, one answers, then swap: **opportunity cost, MB, MC, FC, VC, AC, break-even hours, payback**
- The interactive cards of the page (five per lesson) cover all eight
- The self-checks after each lesson are formative: they do not count toward any grade
- At 1:00 the Brick 3 brief starts

One line each

Opportunity cost = value of the next-best use

MB = min saved ÷ 60 × rate × runs

MC = paid requests × price + set-up ÷ horizon

FC: no change with quantity; VC: changes with it

AC = TC ÷ runs

Break-even h = price ÷ rate (0.71 h)

Payback = set-up ÷ net saving (1.23 mo)

Watch at home: break-even and payback (under 15 minutes each)

about 10:00 length · tutor2u

Breakeven and Contribution (A-Level, IB and BTEC Business)

Why watch it: Introduces contribution per unit and shows that the break-even quantity equals the fixed costs divided by the contribution per unit, with a worked example. The same arithmetic answers the question of Part 4: how many hours saved justify a paid subscription.

Watch on YouTube Creator page

about 9:00 length · tutor2u

Payback Period (Investment Appraisal)

Why watch it: Defines the payback period as the time the savings of an investment take to recover the initial outlay, with a worked calculation and the limits of the method. It supports the 'months to recover the set-up time' part of this lesson.

Watch on YouTube Creator page

Each link was confirmed online and public on October 8, 2026; lengths from the creators' pages or chapter markers, all under 15 minutes. External videos: no affiliation, their own terms apply (Legal Notice sections 5, 14 and 15). Check any address with VirusTotal before you open it.

Real-life example 3.4: free or paid? The owner's decision candidate

- **Task:** break-even hours of a paid seat, payback of the set-up, make-or-buy over 12 and 24 months, then the 120-word decision candidate
- Break-even 43 min a month per user; measured saving 6.7 h: nine times the break-even
- Payback 1 120 ÷ 911 = 1.23 months; make-or-buy: buy at 12 months, build at 24
- The decision candidate names one routine to automate first (e-mail triage), two later (social posts, daily report), one to leave alone (invoice checks)
- It gives the numbers with their units and sources, and the risks the numbers cannot show: a quota that can change, a small sample, a paid assistant that may not do the task
- **Brick 3:** break-even and payback lines; docs/econ_notes.md (about 120 words); break_even_hours() and payback_months() with their guards

Code 3.4.A: class03_break_even_payback.py, step by step

1. Lines 1 to 22 state the task, the link with the lecture and the data: the pilot rows give the saving; the three given numbers and stated assumptions give the rest.

2. The settings: hourly value, 20 $ per user, five users, 20 working days, the 40 set-up hours, one maintenance hour a month, the two review minutes per run, and the file name.

3. find_file() and measured_saving() are the same helpers as in Code 3.2.A: locate the file, compare the pilot participants with themselves.

4. break_even_hours() divides the monthly price by the hourly value and raises a ValueError when the hourly value is not positive; payback_months() divides the set-up cost by the monthly net saving and returns math.inf when nothing is saved, the documented answer for 'never'.

5. Step 1 prints the break-even: 0.71 h = 43 min per user per month, 3.57 h for five users; the visualization draws the gauge.

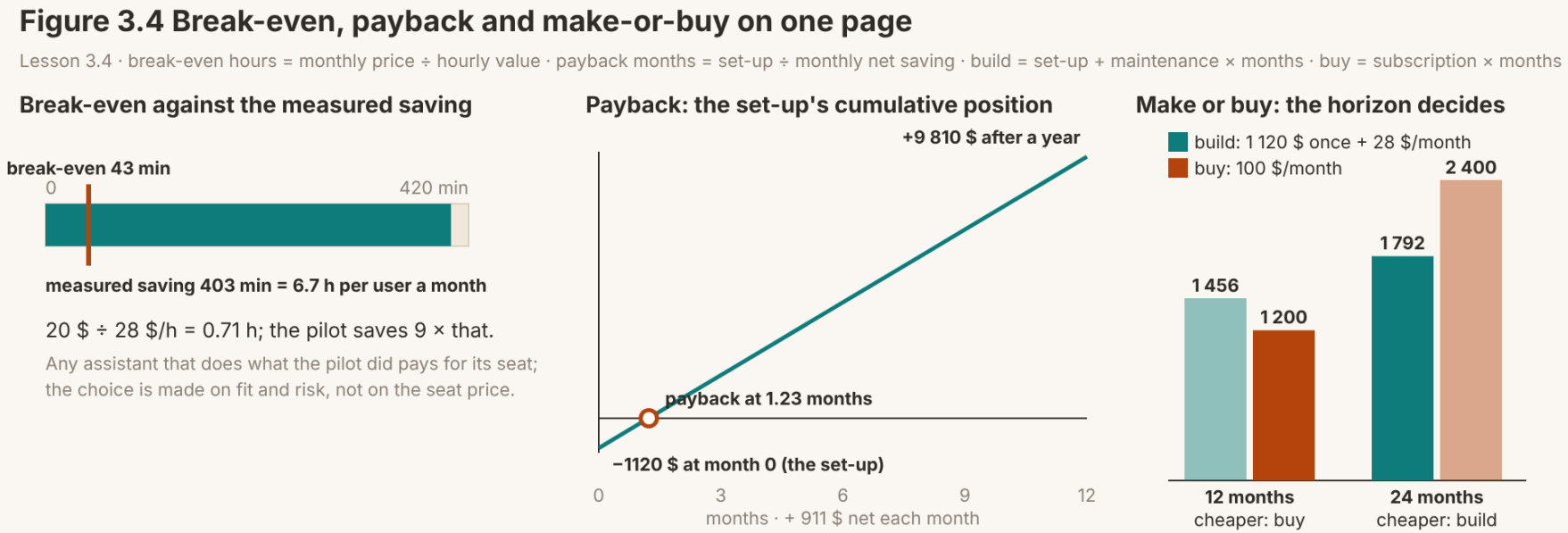
6. Step 2 measures the saving from the file (22.1 minutes, 20.1 net of review) and converts it to hours per user per month (6.7 h): nine times the break-even.

7. Step 3 computes the set-up cost (1 120 $), the monthly net saving (939 − 28 = 911 $) and the payback (1.23 months), then the cumulative position from month 0 (−1 120 $) to month 12; the visualization draws the line crossing zero.

8. Step 4 compares build (set-up + maintenance × months) and buy (subscription × months) at 12 and at 24 months: buy wins at 12, build wins at 24; the sentence 'the horizon decides' is printed.

9. Step 5 assembles the decision candidate from the computed numbers, prints it with its word count, and the run ends. Replace the assumptions with your team's numbers and the text updates itself: that is the shape of docs/econ_notes.md.

Figure 3.4: break-even, payback and make-or-buy on one page



Every formula reads a column, a given number or a stated assumption, and becomes one EconEngine method

EconEngine method	formula	columns · given numbers · assumptions
`routine_cost()`	minutes ÷ 60 × hourly value × runs per month	minutes, routine, staff_id, date · 28 $/h
`opportunity_cost()`	hours × rate of the next-best use	hours from minutes · billable rate (stated)
`marginal_benefit()`	minutes saved ÷ 60 × hourly value × runs	minutes, tool (manual vs jarvis) · 28 $/h
`total_cost / average_cost()`	FC + VC · TC ÷ runs	quota and price (given/stated) · runs from date
`break_even_hours()`	monthly price ÷ hourly value	20 $/user/month · 28 $/h
`payback_months()`	set-up cost ÷ monthly net saving	set-up hours (logged in Phase 2) · MB − running cost

Figure 3.4 Left: the break-even of a paid seat (43 minutes a month) against the measured saving of the pilot (6.7 hours). Centre: the cumulative position of the set-up, from −1 120 $ at month 0 to the payback at 1.23 months. Right: build against buy over 12 and 24 months; the horizon decides. Numbers from Code example 3.4.A.

- Left: break-even 43 min against the measured 403 min
- Centre: cumulative position crossing zero at 1.23 months
- Right: build vs buy at 12 and 24 months
- Bottom: the formula card

CS 251 Integrative Project 1 · Class 3 · Cyber.SoHo Educational Hub · © 2026

Practice 3.4: flash cards and self-check (formative)

Flash cards: can you answer without looking?

- 1. Write the break-even hours formula and compute it for the paid tier.
- 2. Write the payback formula and compute it for the e-mail tool.
- 3. What is a make-or-buy decision, and what decides it here?
- 4. Which columns of the file feed the formulas of this class?
- 5. Name the seven EconEngine methods whose signatures are due with Brick 3.

Self-check on the page: 4 MCQ + 2 MSQ (formative, not graded)

- Q1 MCQ: A paid assistant costs 20 $ per user per month and the hourly value is 28 $. A u…
- Q2 MCQ: The set-up of a tool cost 1 120 $ and the tool brings a net saving of 400 $ a mo…
- Q3 MCQ: Build costs 1 120 $ once plus 28 $ a month; buy costs 100 $ a month. Over 12 mon…
- Q4 MCQ: Which column of boreal_routines_2026.csv separates the pilot runs of the assista…
- Q5 MSQ (select 3): Which three statements about the EconEngine signatures due with Brick 3 are corr…
- Q6 MSQ (select 2): Which two limits must the decision candidate state? (Select 2)

Brick 3, Economic Analysis Script: what the next two hours produce (summary)

- The Brick brief, the script template (analysis/econ_sheet.py, standard library only, the file already loaded) and the checkpoint sheet are posted separately on the LMS
- 1:10–1:50 the routine bill: per routine and per staff member, runs, minutes, monthly cost, the studio bill, the share of each routine (Lesson 3.1)
- 1:50–2:20 automation economics for ≥ 2 routines: minutes saved, MB, set-up cost, MC at the free tier and at the paid price, net benefit, rank (Lessons 3.2–3.3)
- 2:20–2:40 break-even hours, payback months, one routine first / later / never, the 120-word decision candidate in docs/econ_notes.md (Lesson 3.4)
- 2:40 checkpoint: break-even hours called out, range compared; commit the script, its output (docs/econ_output.txt) and the notes
- At home: the seven EconEngine signatures with docstrings; L&O chapter 4; Weekly Progress Report #3
- **Due Thursday, October 15, 2026, 23:59, LMS + GitHub** · feeds §3 Method (economics) and the Mid-Term, Part A



What to remember from Class 3

1. An hour of staff time has a value even when nobody pays for it: the loaded cost (28 $/h) is what the hour costs; the opportunity cost is the value of its next-best use. Cost of a routine = minutes ÷ 60 × hourly value × runs per month.

2. Decisions are taken at the margin: automate a routine (or add a level) while MB ≥ MC; rank by MB − MC; each added routine returns less than the previous one.

3. Name the cost category with respect to a stated quantity: fixed (subscription per period; set-up, paid once), variable (paid requests; staff minutes per run), total = FC + VC, average = TC ÷ runs, marginal = the change for one more unit, zero inside the free quota.

4. A sunk cost is spent under every alternative and must not influence a decision; the set-up enters MC before the decision and the payback after it, never as a deduction from future savings.

5. Break-even hours = monthly price ÷ hourly value (43 min a month per seat); payback months = set-up ÷ monthly net saving (1.23 months); make-or-buy depends on the horizon, on fit and on the quota risk.

6. Every formula maps to a column (minutes, tool, routine, staff_id, date), to a given number or to a stated assumption, and to one EconEngine method; the Formula Card starts today.



Next week and before it

- **Class 4, Friday, October 16: Statistics Refresher, Letting the Time Log Speak**
- Mean, median, mode; sample variance and SD with n − 1; histograms and outliers; correlation (interruptions vs minutes); before/after comparisons
- What 14 days and 5 people can and cannot prove; Brick 4 closes Phase 1
- Bring your own 14-day log, closed on October 8: Class 4 runs the same statistics on it
- Before Class 4: Brick 3 by Thursday 23:59; the seven EconEngine signatures; L&O chapter 4; the eight flash cards; your Formula Card started

Links used in this class (all addresses on the page, with a VirusTotal check button)

- **Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy, v2.0**: https://cyber-soho.github.io/prevmon/00.01.%20cyber.soho%20-%20legal.html
- **Cyber.SoHo Educational Hub, published course pages**: https://cyber-soho.github.io/prevmon/
- **Criminal Code of Canada (R.S.C. 1985, c. C-46), consolidated text**: https://laws-lois.justice.gc.ca/eng/acts/C-46/
- **OpenStax, Principles of Microeconomics 3e (free textbook)**: https://openstax.org/details/books/principles-microeconomics-3e
- **Khan Academy, Microeconomics**: https://www.khanacademy.org/economics-finance-domain/microeconomics
- **Marginal Revolution University, Marginal thinking and the sunk cost fallacy**: https://learn.mru.org/lessons/marginal-thinking-and-sunk-cost-fallacy
- **tutor2u Business, Break-even point and Payback period**: https://www.tutor2u.net/business/reference/breakeven-point
- **Statistics Canada, Earnings, wages and non-wage benefits**: https://www150.statcan.gc.ca/n1/en/subjects/labour/earnings_wages_and_nonwage_benefits
- **Python documentation, the csv module**: https://docs.python.org/3/library/csv.html
- Full list, notes and the videos: the Links section of the lecture page; check any address before you click (Legal Notice, section 14 and Annex C)

Legal notice and disclaimer

- Educational material — Cyber.SoHo / Asen Grozdanov. © 2026 Cyber.SoHo. All rights reserved. Educational content only; no professional, legal or financial advice (section 16)
- Use of this deck, the page, the code and the files is subject to the Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy, v2.0
- Licence and restrictions (sections 4.2, 4.3); acceptable use (section 6); personal responsibility (section 10); no warranties, limited liability (sections 11, 12)
- Data and privacy: a personal time log is personal information; anonymized, private repository, aggregated numbers only (sections 6.7, 8.3, 20); real organizations: section 7.9
- Academic integrity: Brick 3 is the team's own work; cite reused code, documentation pages and AI assistance in APA 7 (section 18); recording and sharing: section 17
- No affiliation with any vendor, platform or institution (section 5); videos and external pages belong to their creators (sections 4.6, 14, 15)
- Students sign Annex A once per course; the e-signature page is provided separately. Fictional company, synthetic data.